

UNIVERSITY LAB REPORT

Department of Electrical Engineering (Communications)

Student Name:	SHAH KHALID	Lab No:	Lab 12
Reg. No:	0672	Subject:	Programming Fundamentals (C++)
Department:	Electrical (Communications)	Date:	May 2026

Objectives

The objective of Lab 12 is to understand and practice the concept of pointers in C++. In Task 1, students will learn how to declare a pointer, assign it the address of a variable, and use the dereference operator to access the stored value. In Task 2, students will explore pointer arithmetic by using a pointer to traverse and display all elements of an integer array. Pointers are one of the most powerful and fundamental features of C++ programming.

Task 1: Pointer Declaration and Initialization

Description: An integer variable num is declared and initialized. A pointer variable ptr is then declared and assigned the memory address of num using the address-of operator &. The dereference operator * is used to access the value stored at the address held by the pointer.

■ C++ Source Code

```
// Standard input/output library include kar rahe hain
#include <iostream>

// std namespace use kar rahe hain taake cout seedha likh sakein
using namespace std;

int main() {

    // Integer variable 'num' declare karke usmein 75 store kar rahe hain
    int num = 75;

    // Integer pointer 'ptr' declare kar rahe hain
    int* ptr;

    // 'ptr' ko 'num' ki memory address assign kar rahe hain (& = address-of operator)
    ptr = &num;

    // 'num' ki direct value print kar rahe hain
    cout << "Value of num (direct) : " << num << endl;
```

```
// Pointer ki madad se num ki value print kar rahe hain (* = dereference operator)
cout << "Value via pointer (*ptr) : " << *ptr << endl;

// Pointer mein stored memory address print kar rahe hain
cout << "Address stored in ptr : " << ptr << endl;

// Num ki actual memory address print kar rahe hain
cout << "Address of num (&num) : " << &num << endl;

return 0; // Program successfully khatam ho gaya
}
```

■ Program Output

```
Value of num (direct) : 75
Value via pointer (*ptr) : 75
Address stored in ptr : 0x61ff08 (address may vary on each run)
Address of num (#) : 0x61ff08 (address may vary on each run)
```

Explanation: The & operator returns the memory address of a variable. When we assign this address to a pointer, the pointer 'points to' that variable. Using *ptr (dereference) gives us the value stored at that address, which is the same as num. Both address outputs are identical, confirming the pointer correctly points to num.

Task 2: Pointer Arithmetic

Description: An integer array numbers[] is declared with five values. A pointer ptr is initialized to point to the first element of the array. Pointer arithmetic (ptr++) is then used inside a loop to move the pointer forward through each element and display all array values without using array indexing.

■ C++ Source Code

```
// Input/output ke liye header file include kar rahe hain
#include <iostream>

// Standard namespace declare kar rahe hain
using namespace std;

int main() {

// 5 integers ka array 'numbers' declare aur initialize kar rahe hain
int numbers[] = {10, 20, 30, 40, 50};

// Array ki total size calculate kar rahe hain (bytes / ek element ki size)
int size = sizeof(numbers) / sizeof(numbers[0]);

// Pointer 'ptr' ko array ke pehle element ki taraf point kara rahe hain
int* ptr = numbers; // numbers array ka naam khud pehle element ka address hai

// Screen par heading print kar rahe hain
cout << "Array elements using pointer arithmetic:" << endl;
cout << "-----" << endl;
```

```
// Loop chala rahe hain array ke har element tak pahunchne ke liye
for (int i = 0; i < size; i++) {
    // *ptr se current element ki value print kar rahe hain
    cout << "Element [" << i << "] = " << *ptr;

    // Pointer ka address bhi print kar rahe hain
    cout << " (Address: " << ptr << ")" << endl;

    // ptr++ se pointer ko agle element ki taraf move kar rahe hain
    ptr++; // Yeh pointer ko 4 bytes (int size) aage le jaata hai
}

return 0; // Program successfully complete ho gaya
}
```

■ Program Output

```
Array elements using pointer arithmetic:
-----
Element [0] = 10 (Address: 0x61fef4)
Element [1] = 20 (Address: 0x61fef8)
Element [2] = 30 (Address: 0x61fefc)
Element [3] = 40 (Address: 0x61ff00)
Element [4] = 50 (Address: 0x61ff04)
```

Explanation: In C++, an array name itself holds the address of its first element. When a pointer is incremented using `ptr++`, it does not simply add 1 to the address — it moves forward by the size of the data type (`sizeof(int) = 4` bytes). This is called pointer arithmetic. Notice each address in the output increases by 4, confirming this behaviour. This technique allows efficient traversal of arrays without subscript notation.

Key Concepts Summary

Operator / Concept	Symbol	Description
Address-of Operator	<code>&</code>	Returns the memory address of a variable
Dereference Operator	<code>*</code>	Accesses the value stored at a pointer's address
Pointer Increment	<code>ptr++</code>	Moves pointer to next memory location (by data
Array-Pointer Relation	<code>arr == &arr[0]</code>	Array name is the address of its first element

Conclusion

In Lab 12, the concept of pointers in C++ was studied and implemented through two practical tasks. Task 1 demonstrated how to declare a pointer, assign it a variable's address using the `&` operator, and retrieve the value using the dereference operator `*`. Task 2 extended this knowledge by

applying pointer arithmetic to iterate over an integer array, showing how incrementing a pointer moves it by the size of its data type in memory. All programs were written with line-by-line Urdu comments (in English letters) for better understanding. Pointers provide direct memory access and are essential for advanced C++ programming including dynamic memory allocation and data structures.

Student Signature: _____

Instructor Signature: _____